



COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

WARNING

This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the *Copyright Act 1968* (the Act).

The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice.



FIT3080 – Artificial Intelligence

Solving Problems as Search – Problem Formulation and Uninformed Search

Sections 3.1-3.4.4, 3.4.6, 5.1, 5.3, 5.4

Why search?

- Effective at solving sequential problems
- Easily integrated with domain-specific knowledge
- Some applications of search:



Video games



Task and motion planning



Scheduling



LN3 Learning Objectives

- **Problem formulation**
- **Problem-solving agents**
- **Search algorithms and strategies**
 - **Tentative algorithms**
 - > Backtrack [Chapter 5.1, 5.3, 5.4]
 - > Tree and graph search **strategies**:
 - **Uninformed**: BFS, UCS, DFS, DLS, IDS [Chapter 3.1-3.4.4, 3.4.6]
 - **Informed**: Greedy best-first search, A, A* [Chapter 3.5.1-3.5.4]
 - **Irrevocable algorithms**
 - > **Informed**: Hill climbing, Local beam search, Simulated annealing, Genetic algorithms [Chapter 4.1]
- **Adversarial search algorithms [Chapter 6.1-6.3]**
 - Optimal decisions
 - Minimax, α - β pruning



Assumptions about the Environment

- **Fully observable**
- **Known**
- **Single/multi agent**
- **Deterministic**
- **Sequential/episodic**
- **Static/dynamic**
- **Discrete/continuous**





FIT3080 – Artificial Intelligence

Problem Formulation

Definitions

- **State: a situation in the environment**
 - **State space:** the set of all possible states
 - **Search space:** the set of states reachable by a search algorithm
 - **Initial state:** the starting state of the agent
 - **Goal state(s):** the desired state(s)
- **Action: given a state, an operation that the agent can do**
 - **Action space:** set of all actions
- **Transition function: given a state and an action, the set of possible states resulting from taking the action**
 - **Cost/reward function:** the amount the agent is penalized or rewarded for a transition
- **Solution/path: a sequence of actions from the initial state to a goal state**
 - Has a path cost

Problem-solving Agents

Function Simple-Problem-Solving-Agent(*percept*)
returns *seq*

persistent: *state* – description of current world state
seq – action sequence
goal – a goal
problem – a problem formulation

} initially null

state $\leftarrow$ UpdateState(*state*, *percept*)
goal $\leftarrow$ FormulateGoal(*state*)
problem $\leftarrow$ FormulateProblem(*state*, *goal*)
seq $\leftarrow$ Search(*problem*)
return *seq*

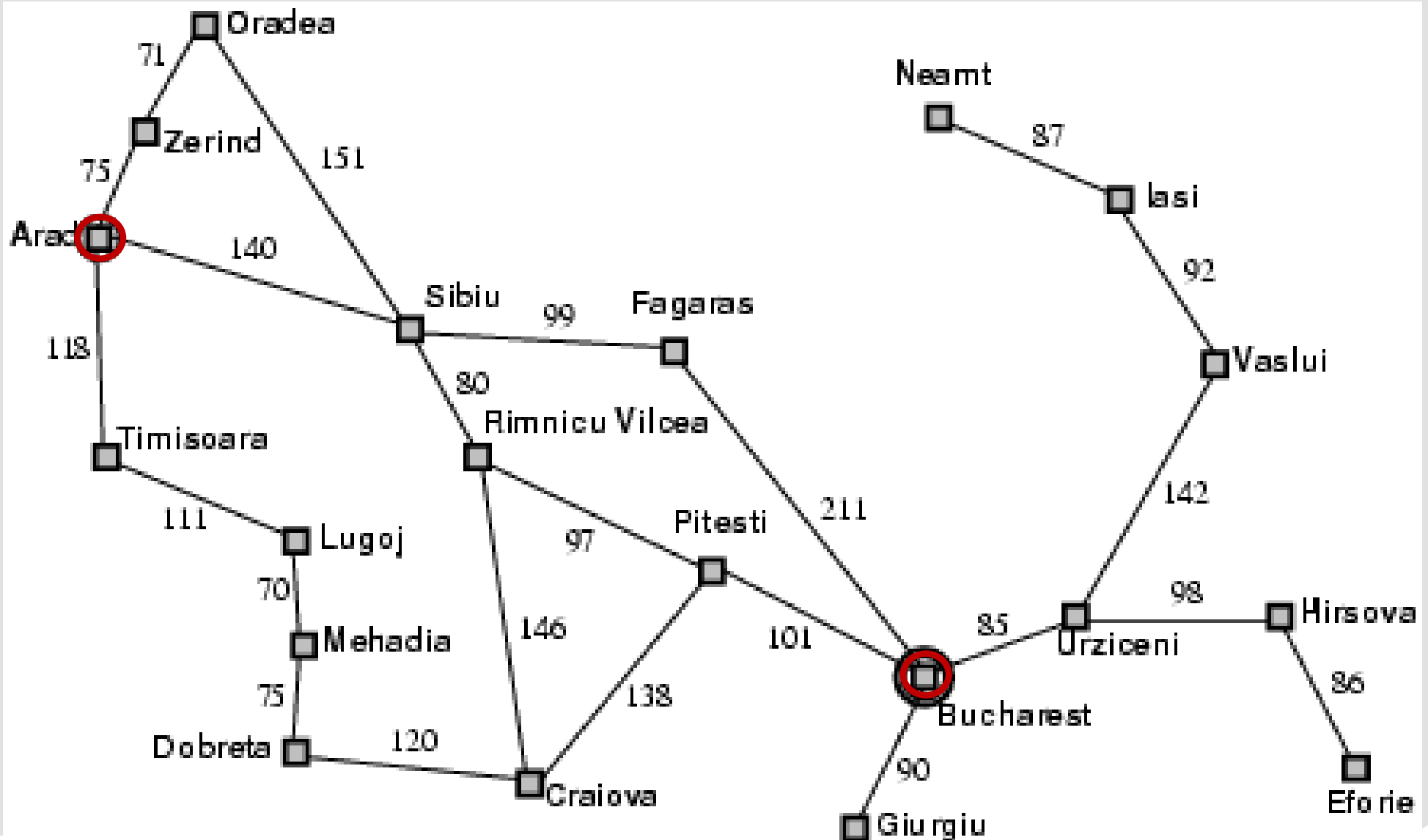


Problem Formulation

- **Problem representation**
 - States – include initial state
 - Goal(s) – requires a goal test
 - Operators (Actions) and transition function
 - > include cost information
 - Constraints
- **Need to make decisions about**
 - Which properties of the world matter?
 - Which actions are possible and relevant?
 - How to represent world states and actions?

Abstracting away from unnecessary detail can drastically reduce the size of the state/search space

Example: Romania Problem



Problem Formulation – Romania (I)

On holiday in Romania; currently in Arad. Flight leaves tomorrow from Bucharest.

- **State:**
 - Romania
 - **UpdateState** → in Arad
- **Formulate goal:**
 - be in Bucharest
- **Formulate problem:**
 - **states**: various cities
 - **actions**: drive between cities
- **Find solution:**
 - sequence of cities, e.g., Arad, Sibiu, Fagaras, Bucharest

Problem Formulation – Romania (II)

1. initial state

- e.g., $\text{In}(\text{Arad})$

2. actions

- e.g., $\{\text{Go}(\text{Sibiu}), \text{Go}(\text{Timisoara}), \dots\}$

transition function

- e.g., $\text{Result}(\text{In}(\text{Arad}), \text{Go}(\text{Zerind})) \rightarrow \text{In}(\text{Zerind})$

3. constraints – nil

4. goal test can be

- explicit, e.g., $\text{In}(\text{Bucharest})$
- implicit, e.g., $\text{Distance}(\text{location}, \text{Airport}) < 5 \text{ Km}$

5. path cost (additive)

- e.g., sum of distances, number of actions executed
- $c(s, a, s')$ is the step cost of taking action a at state s to reach state s' (assumed to be ≥ 0)

Problem Formulation – 8 Puzzle (I)

Start

5	4	
6	1	8
7	3	2

End

1	2	3
8		4
7	6	5

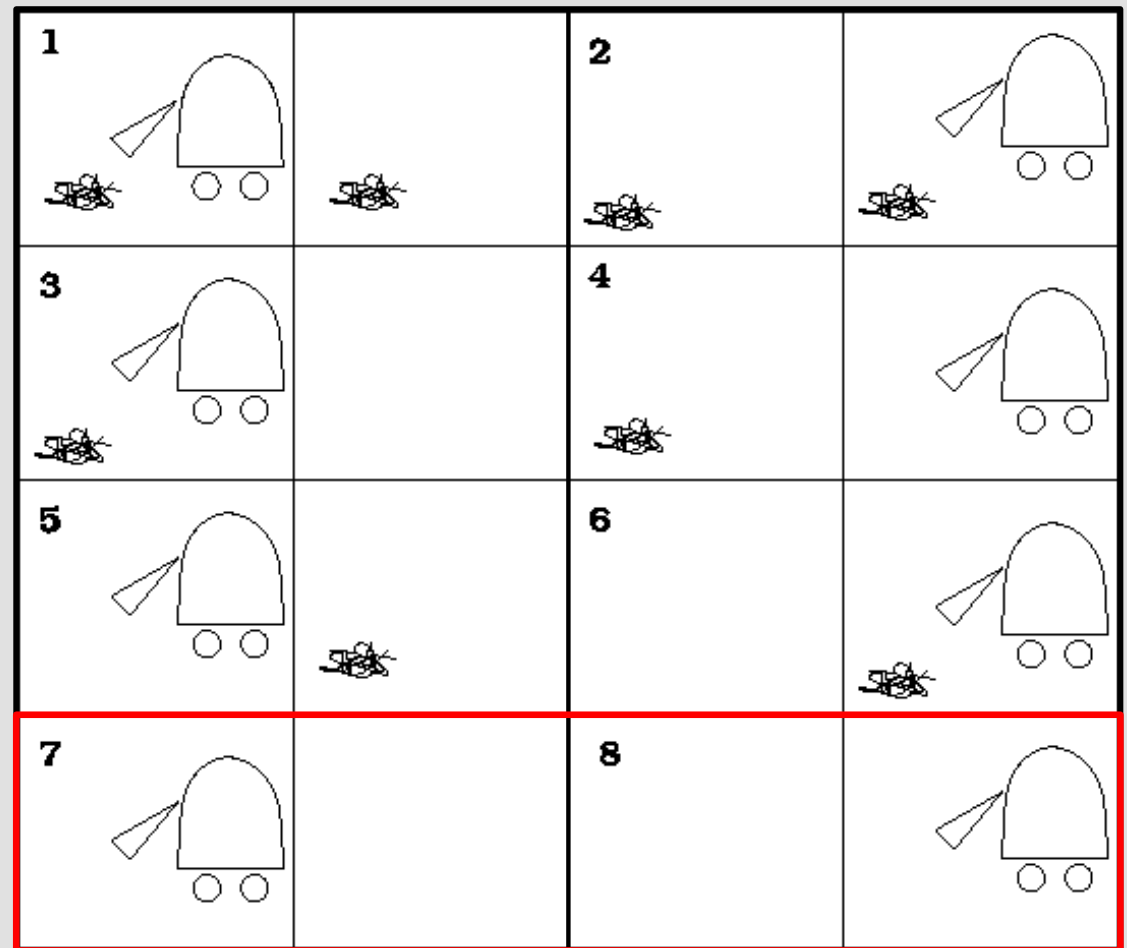


Problem Formulation – 8 Puzzle (II)

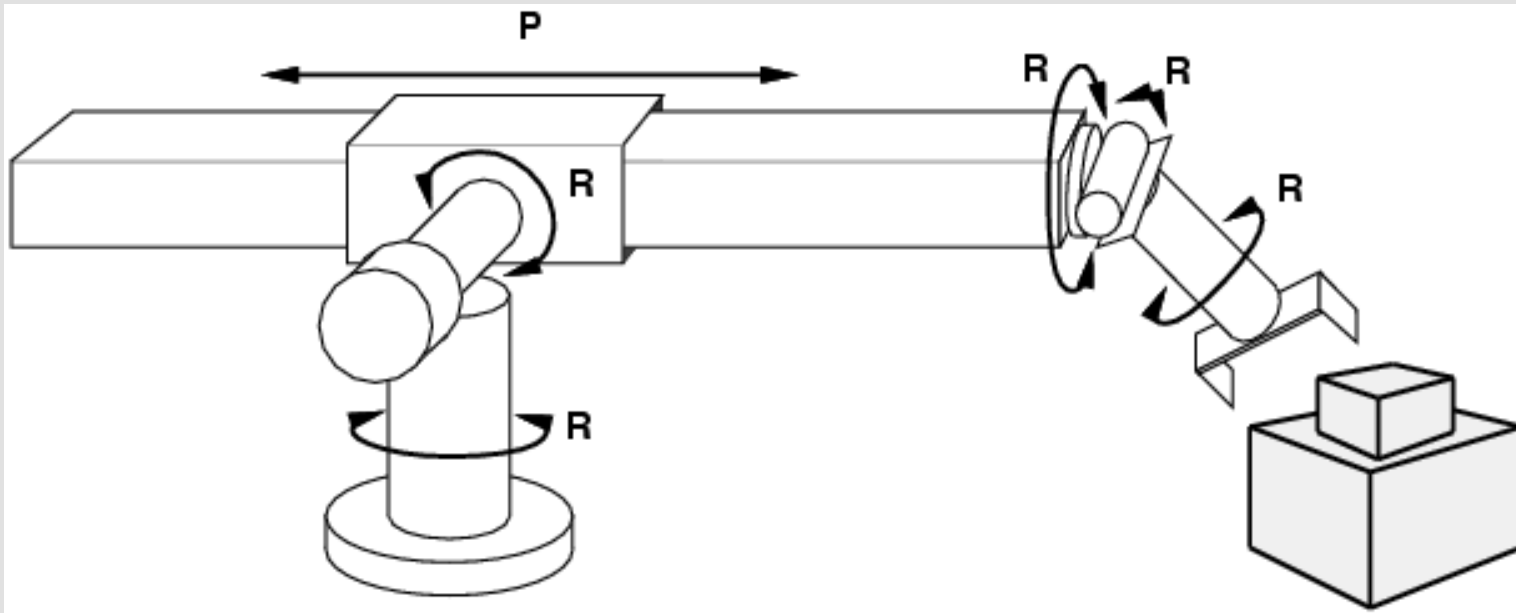
- **States**
 - Location of each of the 8 tiles in one of the 9 squares
- **Operators**
 - Possible moves of the blank tile
- **Constraints**
 - A tile cannot move out of bounds
- **Goal test**
 - Have we reached the goal configuration?
- **Path cost**
 - If we want to minimize the number of steps, then all steps have the same cost (usually 1)

Problem Formulation – Vacuum World

- **States**
 - 8 states shown
- **Operators**
 - Left, right, suck
- **Constraints**
 - Can't leave the space
- **Goal test**
 - States 7 and 8
- **Path cost**
 - Each action costs 1



Problem Formulation: Robotic Assembly



- states?: real-valued coordinates of robot joint angles; parts of the object to be assembled
- actions?: continuous motions of robot joints
- constraints?: arm cannot fully rotate up and down
- goal test?: complete assembly
- path cost?: time to execute



Selecting a State Space

- **Real world is complex**
 - state space must be **abstracted** for problem solving
- **(Abstract) state = set of real states**
- **(Abstract) action = combination of real actions**
 - e.g., “Arad → Zerind” represents a complex set of possible routes, detours, rest stops, etc
 - Each abstract action should be “easier” to perform than solving the original problem
- **(Abstract) solution = a solution that can be expanded into a set of real paths in the real world**
- **Guaranteed realizability: we must be able to reach **some** real state (“in Zerind”) from **any** real state (“in Arad”)**



FIT3080 – Artificial Intelligence

Search Algorithms

Classification of Search Algorithms

- **Tentativeness**

- Irrevocable – no reconsideration
- Tentative – with reconsideration

- **Informedness**

- Uninformed – decide based **only** on problem definition
- Informed – use guidance on where to look for solutions

	Irrevocable	Tentative
		Backtrack
Uninformed	–	Tree- and Graph-Search (BFS, UCS, DFS, DLS, IDS)
Informed	Hill climbing, Local beam search, Simulated annealing, Genetic algorithms	Greedy best-first search, A, A*, IDA*



FIT3080 – Artificial Intelligence

Tentative Search Algorithms

Tentative Search Algorithms

- **Backtrack** – at any point in time, keep one path only
 - If we fail, we go back to the last decision point and **erase the failed path**
- **Graphsearch** – keep track of several paths simultaneously
 - Done using a structure called a ***search tree / search graph***





FIT3080 – Artificial Intelligence

Backtrack

Basic Backtrack Algorithm

Procedure Backtrack (State)

1. If Goal(State) Then return SUCCEED
2. If Deadend(State) Then return FAIL
3. Operators $\leftarrow$ ApplicableOps(State)
4. Loop
 1. If null(Operators) Then return FAIL
 2. Op $\leftarrow$ Pop(Operators)
 3. State' $\leftarrow$ Op(State)
 4. Path $\leftarrow$ Backtrack(State')
 5. If Path=FAIL Then go Loop
 6. Return {Op, Path}

End



When should we backtrack?

- **Backtracking occurs when**

- we reach a DEADEND state OR
- there are no more applicable rules OR
- we generate a previously encountered state description OR
- an arbitrary number of rules has been applied without reaching the goal

Basic
Backtrack

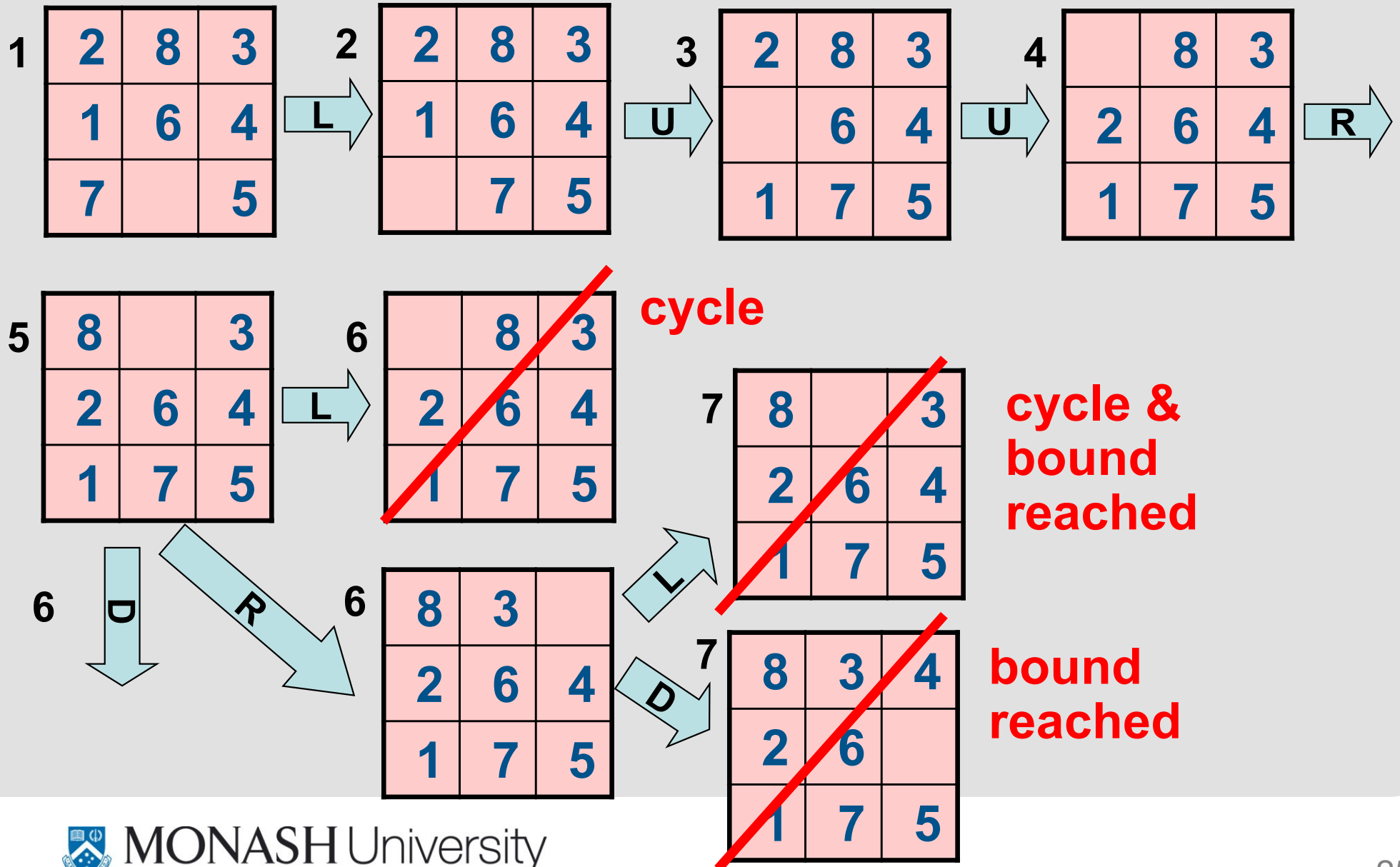
Backtrack1



Backtrack Example – Goal

1	2	3
8		4
7	6	5

Bound = 6



Backtrack Algorithm

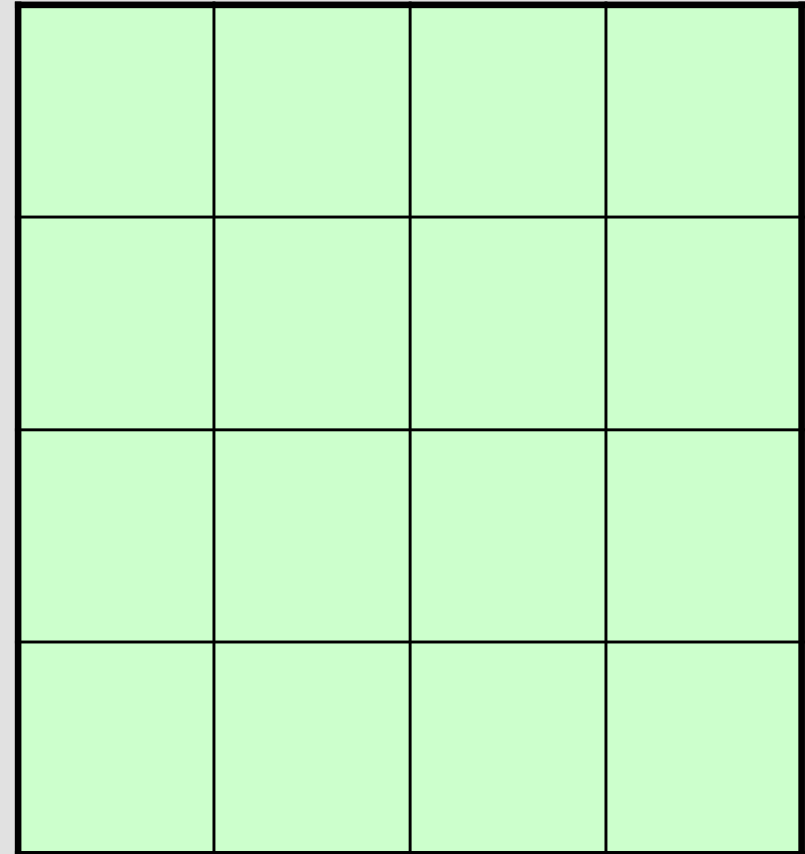
Procedure Backtrack1(StateList)

1. **State** $\leftarrow$ **First(StateList)**
2. **If** **State** $\in$ **RestOf(StateList)** **Then** return **FAIL**
3. **If** **Goal(State)** **Then** return **SUCCEED**
4. **If** **Deadend(State)** **Then** return **FAIL**
5. **If** **Length(StateList) > Bound** **Then** return **FAIL**
6. **Operators** $\leftarrow$ **ApplicableOps(State)**
7. **Loop**
 1. **If** **null(Ops)** **Then** return **FAIL**
 2. **Op** $\leftarrow$ **Pop(Ops)**
 3. **State'** $\leftarrow$ **Op(State)**
 4. **StateList'** $\leftarrow$ **{State', StateList}**
 5. **Path** $\leftarrow$ **Backtrack1(StateList')**
 6. **If** **Path=FAIL** **Then** go **Loop**
 7. **Return** **{Op, Path}**
- End**



Backtracking Example – 4 Queens Problem

- **Start state:**
 - empty chess board
- **Goal state:**
 - 4 queens placed on chess board
- **Constraints:**
 - queens don't attack each other
- **Operators:**
 - place queen on tile (x,y)
- **Path cost: NA**

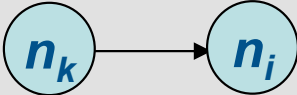




FIT3080 – Artificial Intelligence

TreeSearch and GraphSearch

Graphsearch – Definitions

- **Graphsearch** is a means of finding a path in a graph from a node representing the initial state to a node that satisfies the goal condition
- **Definitions**
 - **Graph** – set of nodes
 - **Arcs** – connect between certain pairs of nodes
 - **Directed graph** – formed by arcs directed from one node to another
 - n_i is a **child** of n_k if  $n_k \rightarrow n_i$
 - n_i is **accessible from** n_k if there is a path from n_k to n_i
 - **Expanding a node** – finding all its children
 - **Search Problem** – find a path between node s and any member of the **goal set** $\{t_i\}$ that represents states satisfying the goal condition



Search Tree

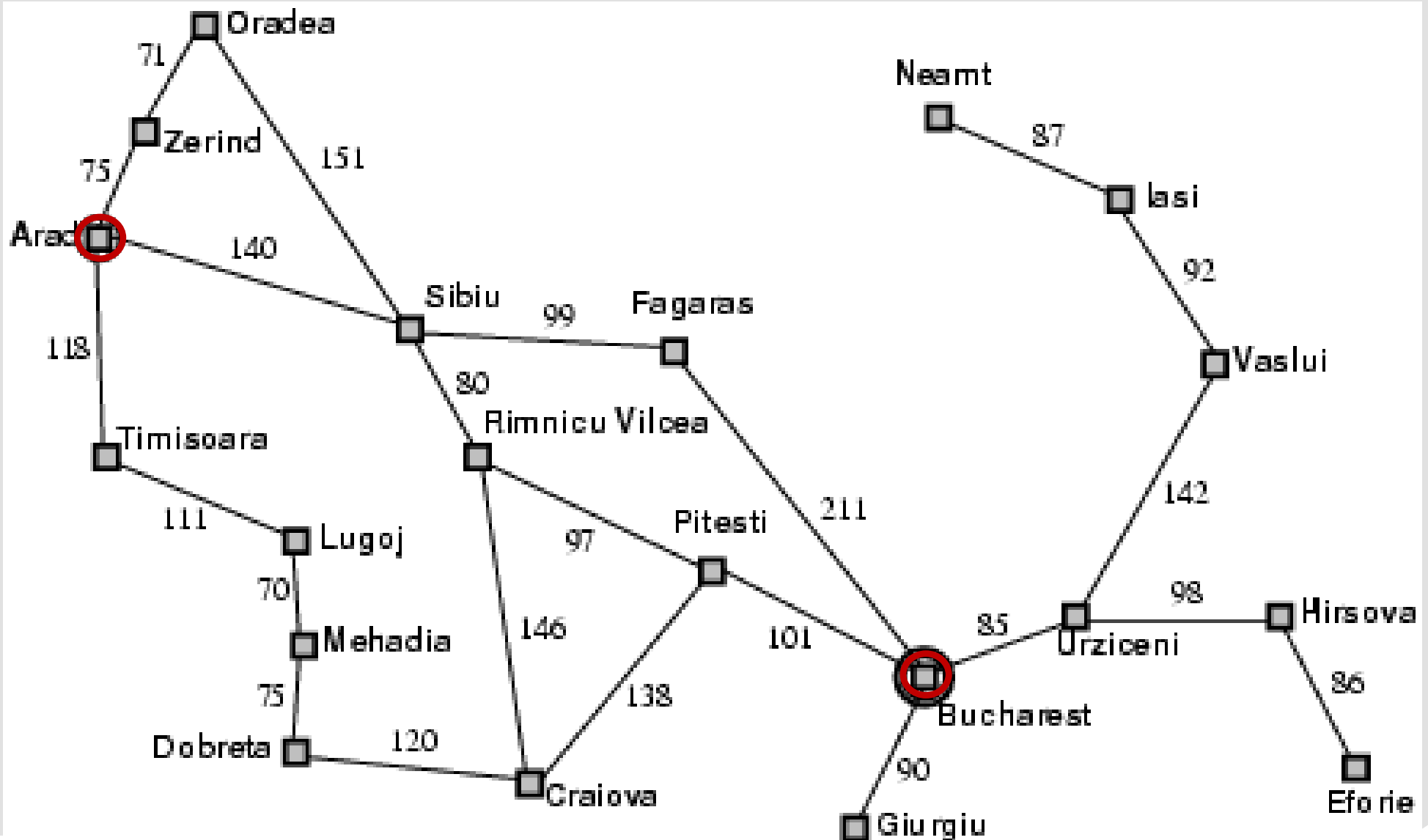
- **Tree** – each node has at most one parent
- **Root** of search tree is the initial state
- **Leaves** are states without successors (“frontier”)
- **At each step, choose one leaf node to *expand***

Basic TreeSearch Algorithm

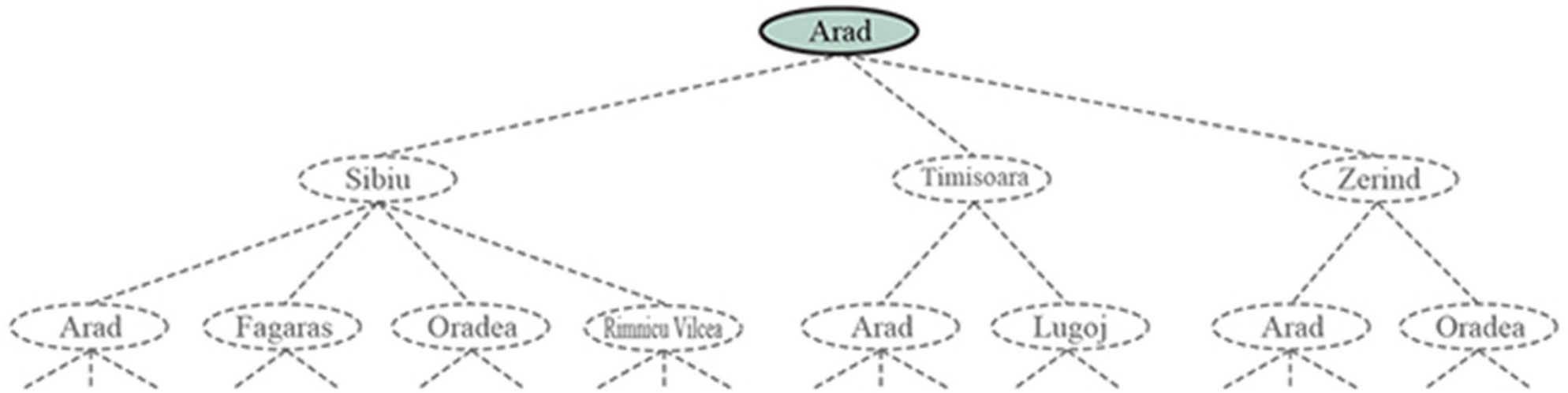
function TREESEARCH(*problem*) **returns** a solution or failure

- Initialize the *frontier* using the initial state of *problem*
- **Loop**
 1. **if** the *frontier* is empty **then**
 return failure
 2. **choose** a leaf node and remove it from the *frontier*
 3. **if** the node contains a goal state **then**
 return the corresponding solution
 4. **expand** the chosen node, **adding** the resulting nodes to the *frontier*
- **End**

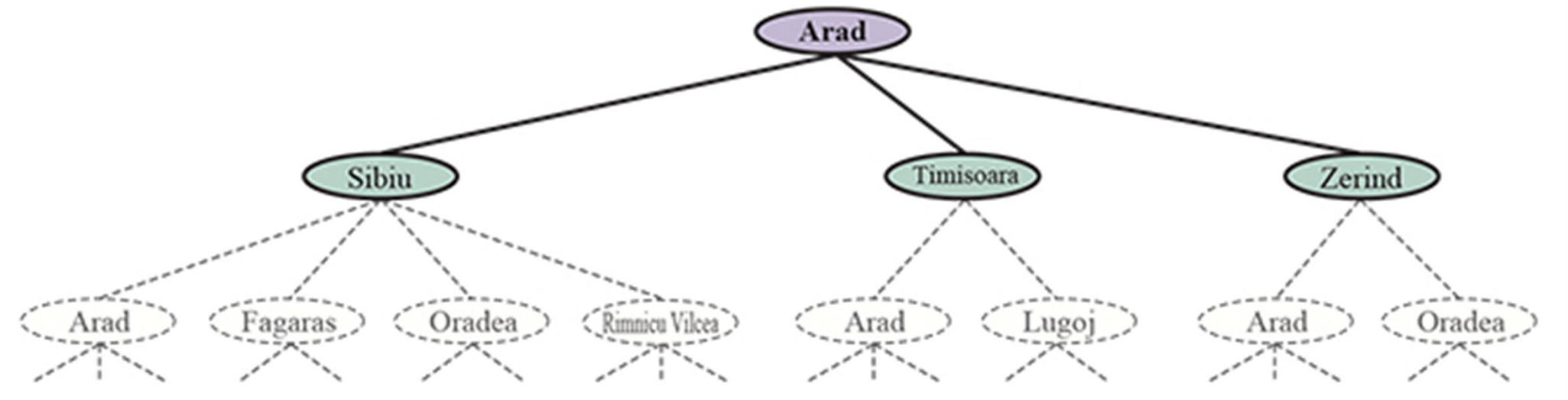
Example: Romania Problem



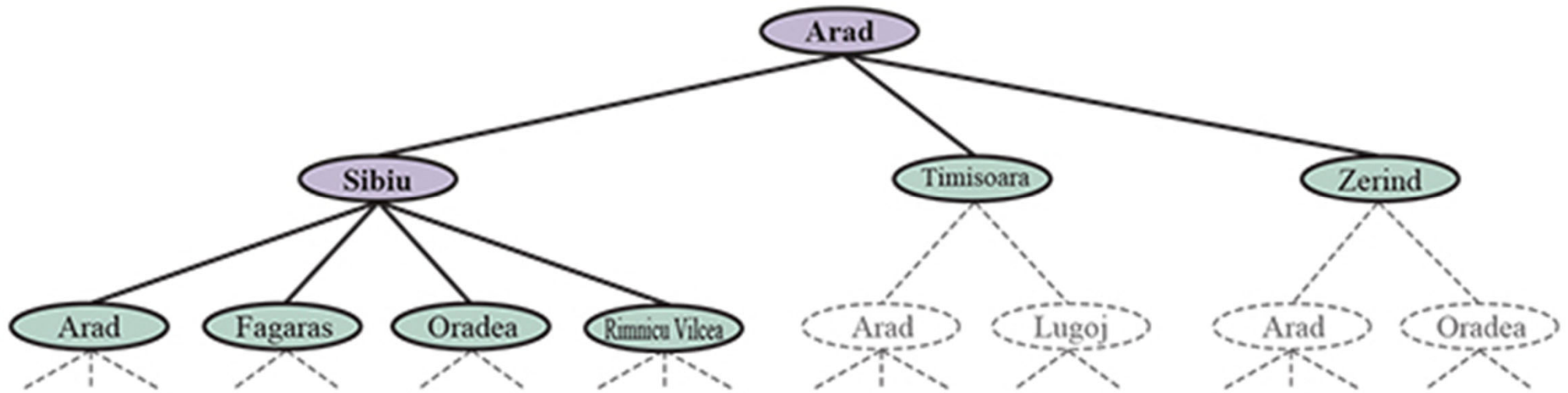
Example: TreeSearch (I)



Example: TreeSearch (II)

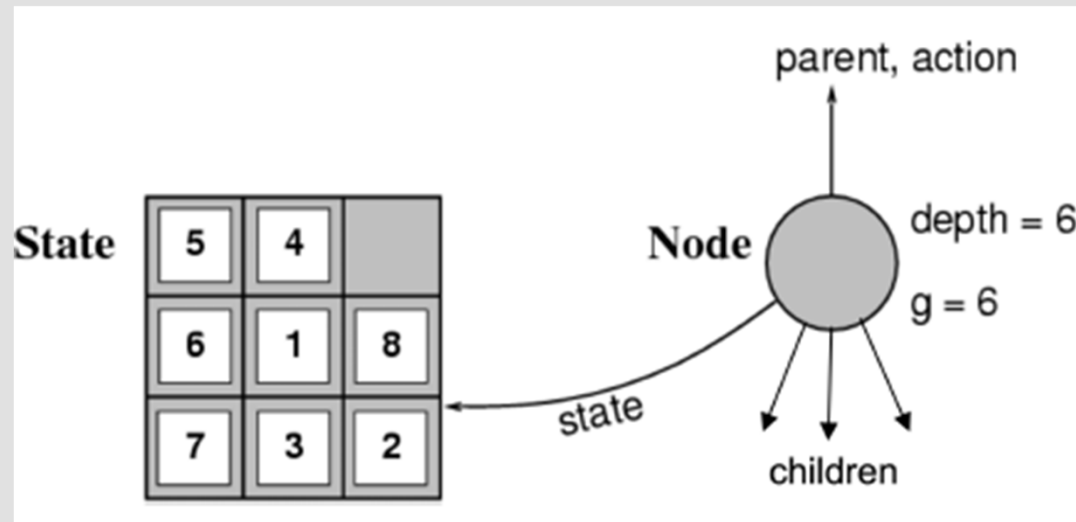


Example: TreeSearch (III)



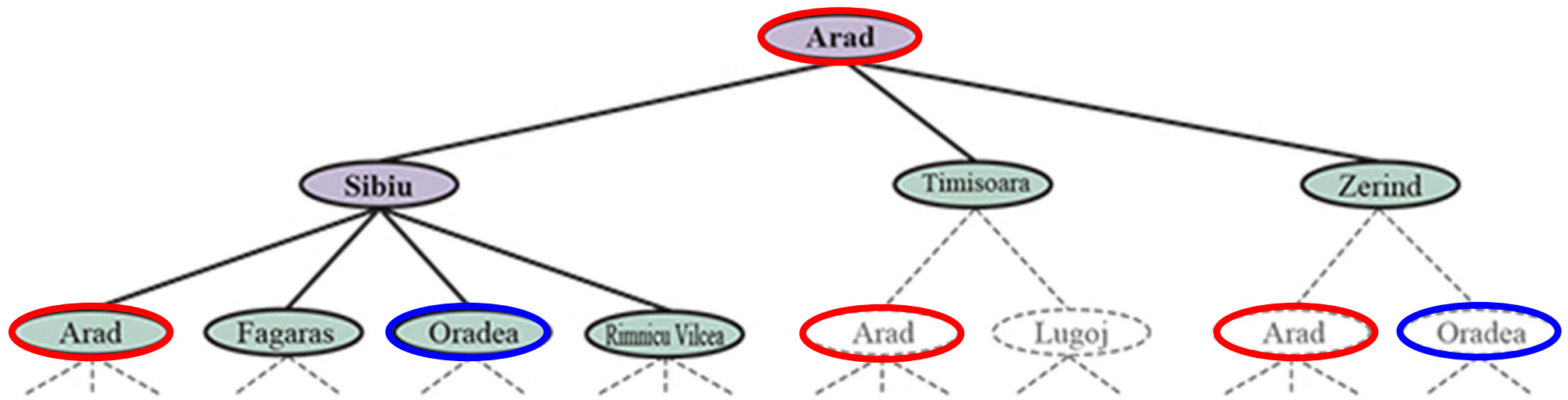
Implementation: States vs. Nodes

- **state** – a (representation of a) physical configuration
- **node** – a data structure that is part of a search tree
 - includes *state*, *parent node*, *action*, *children*, *path cost $g(x)$* , *depth*



- **The *Expand* function**
 - creates new nodes, fills in the various fields
 - uses ***SuccessorFn(Operators)*** to create the corresponding states

Example: Repeated States



Dealing with Repeated States

- **Three ways to deal with repeated states (ordered by cost and effectiveness):**
 - Don't return to the state you just came from
 - Don't generate successors with same state as a node's parent
 - Don't create paths with cycles in them
 - Don't generate successors with same state as any ancestor
 - Don't generate any state that was ever generated before
 - Use hashset (or some way of remembering) to check if a state has been visited

Searching graphs: Multiple Paths to a Node

- **Often search is better represented via graphs:**
 - There may be multiple paths to the same state
 - Improvement (due to search graph) depends on how costly it is to determine a node has already been visited

Basic GraphSearch Algorithm

function GRAPHSEARCH(*problem*) **returns** a solution or failure

- Initialize the *frontier* using the initial state of *problem*
- Initialize the *explored set* to $\emptyset$
- **Loop**
 1. **if** the *frontier* is $\emptyset$ **then**
 return failure
 2. **choose** a leaf node and remove it from the *frontier*
 3. **if** the node contains a goal state **then**
 return the corresponding solution
 4. add the node to the *explored set*
 5. **expand** the chosen node, **merging** the resulting nodes with the *frontier* or the *explored set*

End



Example: TreeSearch (IV)



Basic Search Algorithm: Key Issues

- **Return a path or a node?**
- **Unboundedness:**
 - Tree search: because of loops
 - Graph/tree search: because the state space is infinite
- **Tree search: Repeated states**
 - Failure to detect repeated states can increase the complexity of a problem
- **How are the nodes ordered? → Search strategy**
 - Is the graph weighted or unweighted?
 - What is known about the “quality” of intermediate states?
 - Is the aim to find a *minimal cost path* or *any path asap*?



FIT3080 – Artificial Intelligence

TreeSearch and GraphSearch Strategies

Search Strategies

- A search strategy is defined by picking the **order of node expansion**
- Strategies are evaluated along several dimensions:
 - **completeness**: does it always find a solution if one exists?
 - **time complexity**: maximum number of nodes generated
 - **space complexity**: maximum number of nodes in memory
 - **optimality**: does it always find a least-cost solution?
- **Time and space complexity are measured in terms of**
 - b : maximum branching factor of the search tree
 - d : depth of the least-cost solution
 - m : maximum depth of any path in the search space (may be ∞)





FIT3080 – Artificial Intelligence

Uninformed Search Strategies

Uninformed Search Strategies

Uninformed search strategies choose nodes to expand based only on the information available in the problem definition

- Breadth-first search (BFS)
- Uniform-cost search (UCS)
- Depth-first search (DFS)
- Depth-limited search (DLS)
- Iterative deepening search (IDS)



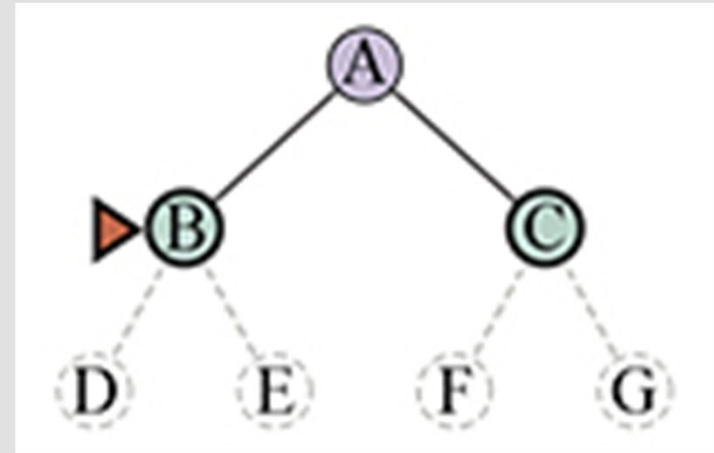
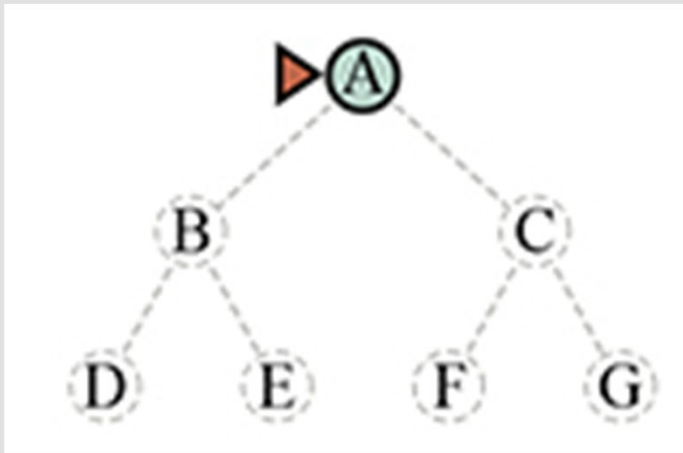


FIT3080 – Artificial Intelligence

Breadth First Search

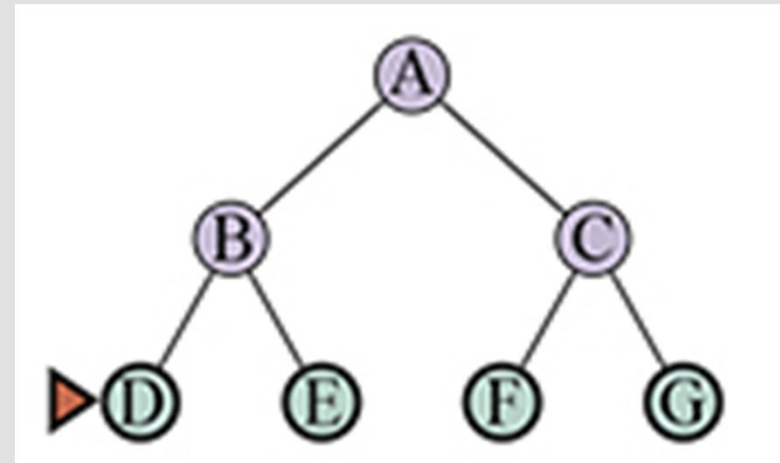
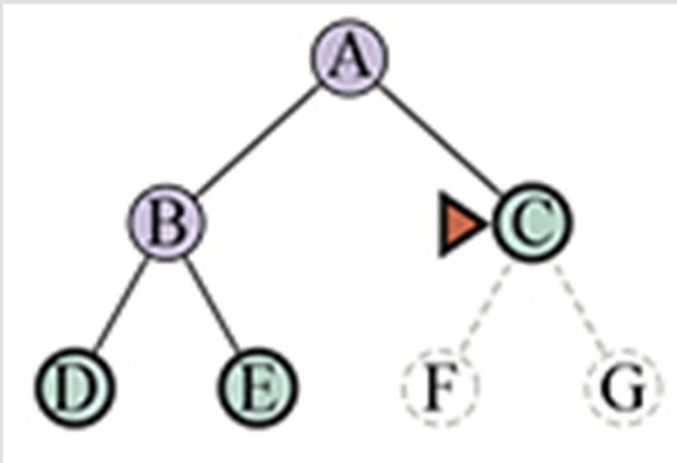
Breadth-first Search (I)

- **Expand the shallowest unexpanded node**
- **Implementation: managing the frontier**
 - QUEUEING-FN: FIFO – put successors at end of queue



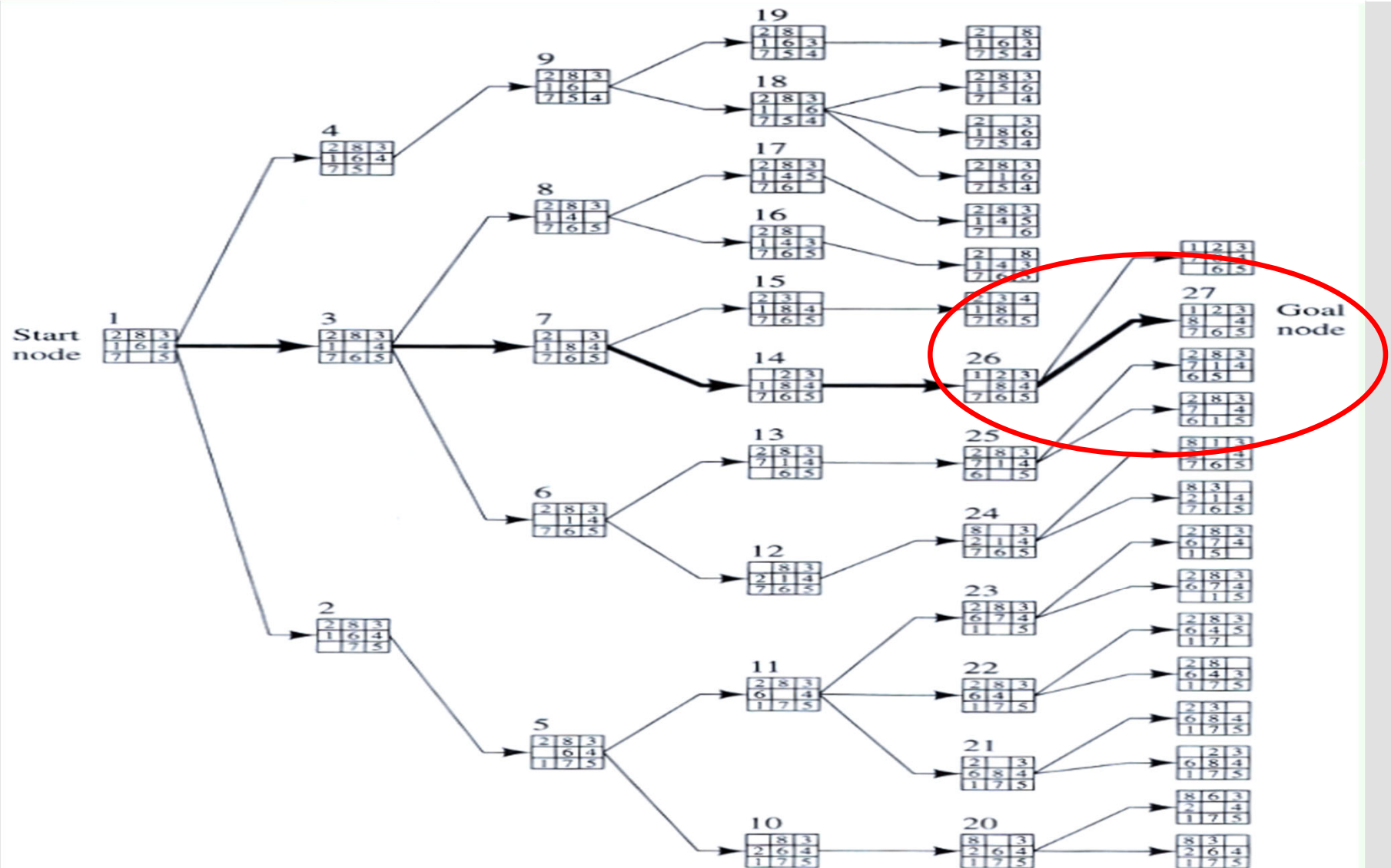
Breadth-first Search (II)

- **Expand shallowest unexpanded node**
- **Implementation: managing the frontier**
 - QUEUEING-FN: FIFO – put successors at end of queue



Only tree/graphsearch algorithm that can stop when the goal node is reached

Breadth-first Search – Example



Properties of Breadth-First Search

- Complete? Yes (if b is finite and the state space is finite)
- Time? $b + b^2 + b^3 + \dots + b^d = b \frac{b^d - 1}{b - 1} \rightarrow O(b^d)$
- Space? $O(b^d)$ (keeps every node in memory)
- Optimal? Yes (if all actions have the same cost)

Space is the bigger problem



Complexity of Breadth-First Search

Depth	Nodes	Time	Memory
2	110	0.11 milliseconds	107 kilobytes
4	11,110	11 milliseconds	10.6 megabytes
6	10^6	1.1 seconds	1 gigabyte
8	10^8	2 minutes	103 gigabytes
10	10^{10}	3 hours	10 terabytes
12	10^{12}	13 days	1 petabyte
14	10^{14}	3.5 years	99 petabytes
16	10^{16}	350 years	10 exabytes

Assumptions: branching factor $b = 10$, 1 million nodes/second, 1000 bytes per node





FIT3080 – Artificial Intelligence

Uniform Cost Search

Uniform-cost Search

- **Expand the least-cost unexpanded node**
- **Implementation: managing the frontier**
 - QUEUEING-FN: insert successors in order of increasing path cost



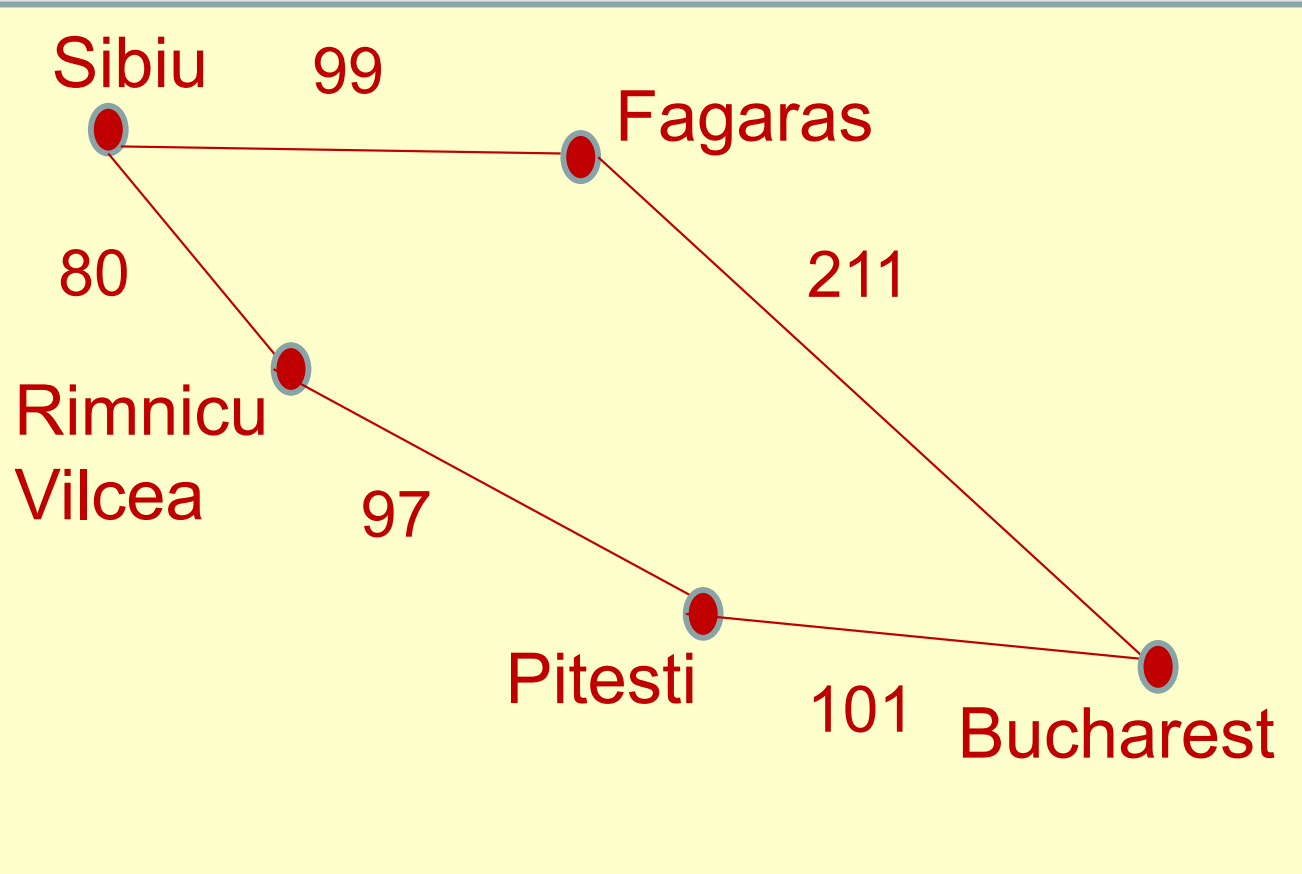
Uniform-Cost Search Algorithm

function UNIFORMCOSTSEARCH(*problem*) **returns** a solution or failure

- Initialize *frontier* using the initial state of *problem*
 - Initialize *explored* to $\emptyset$
 - **Loop**
 1. **if** the *frontier* is $\emptyset$ **then return** failure
 2. **choose** the lowest-cost node in the *frontier*
 - remove it from the *frontier* **and add it to** *explored*
 3. **if** the node contains a goal state **then return** the node
 4. **expand** the chosen node $\rightarrow$ generate its children
 - **if** the resulting nodes are not in the *frontier* **or** *explored* **then add** them to the *frontier*
 - **else if** the resulting nodes are in the *frontier* **with higher path cost**, **then replace** them with the new nodes
- end**

Why aren't we checking nodes in *explored*?

Uniform-cost Search: Example

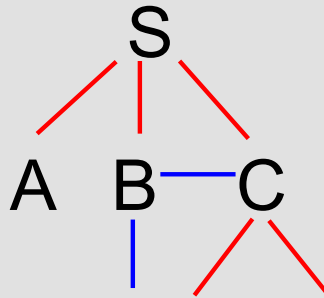


Frontier	Explored
S	



Optimality of Uniform-cost Search

- **Whenever UCS selects a node for expansion, it has found the optimal path to that node**
- **Argument:**



- Say $\text{Cost}(S-C) < \text{Cost}(S-B) < \text{Cost}(S-A)$
→ C is expanded first
- Now say B is expanded, and it has a path to C
 $\text{Cost}(S-B) > \text{Cost}(S-C)$
 $\text{Cost}(S-B) + \text{Cost}(B-C) > \text{Cost}(S-C)$



Properties of Uniform-cost Search

- Complete? Yes, if step cost $\geq \epsilon$ and b is finite
- Time? $O(b^{1+\text{floor}(C^*/\epsilon)})$
 - where C^* is the cost of the optimal solution
- Space? $O(b^{1+\text{floor}(C^*/\epsilon)})$
- Optimal? Yes – nodes expanded in increasing order of the cost of the path to node n

1. When all step costs are the same, UCS does more work than BFS. Why?
2. When UCS selects a node for expansion, the optimal path to that node has been found.
3. Why is UCS an uninformed search strategy?

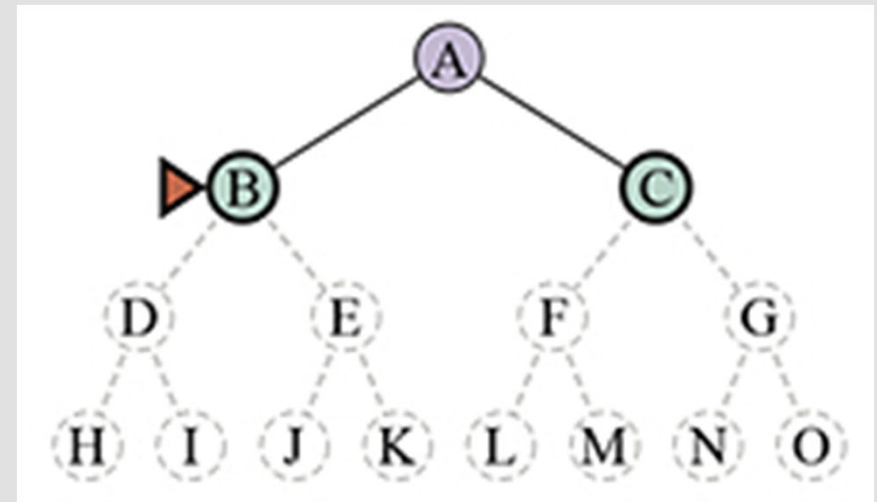
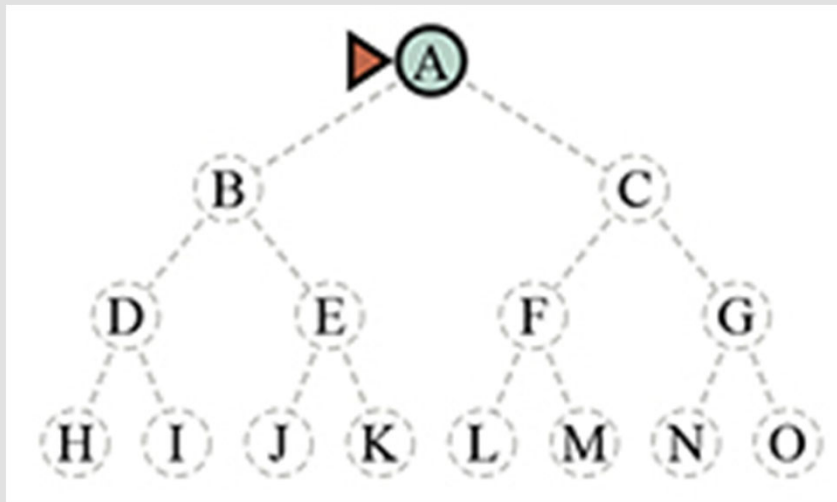


FIT3080 – Artificial Intelligence

Depth First Search and Depth-limited Search

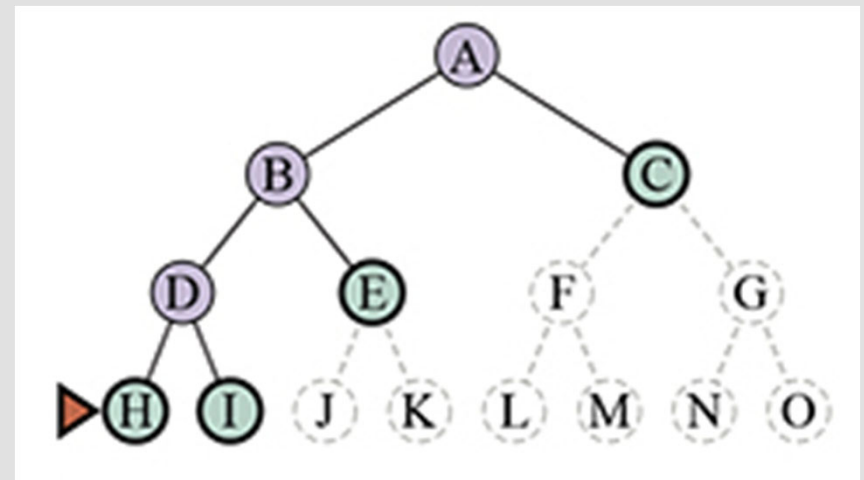
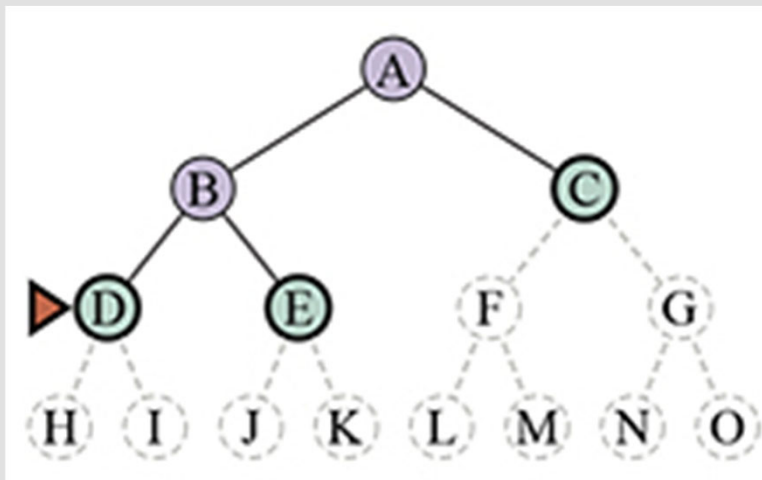
Depth-first Search (I)

- **Expand the deepest unexpanded node**
- **Implementation: managing the frontier**
 - QUEUEING-FN: LIFO – insert successors in front of queue



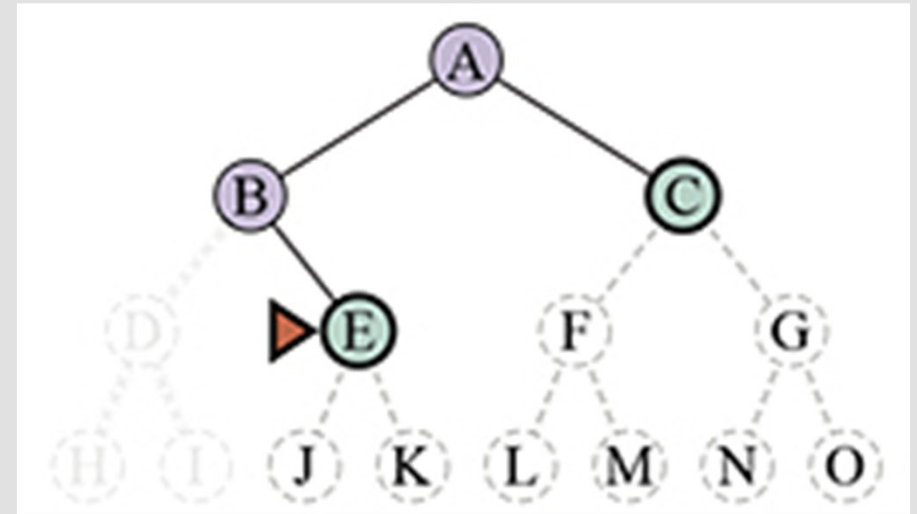
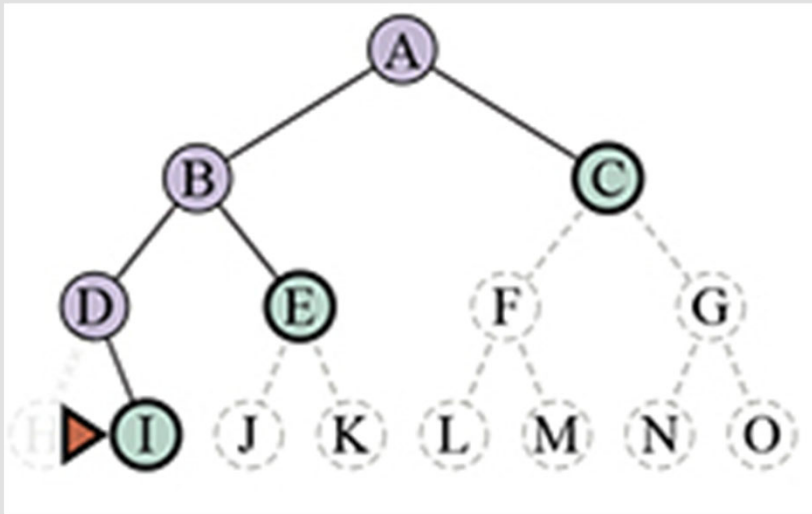
Depth-first Search (II)

- **Expand the deepest unexpanded node**
- **Implementation: managing the frontier**
 - QUEUEING-FN: LIFO – insert successors in front of queue



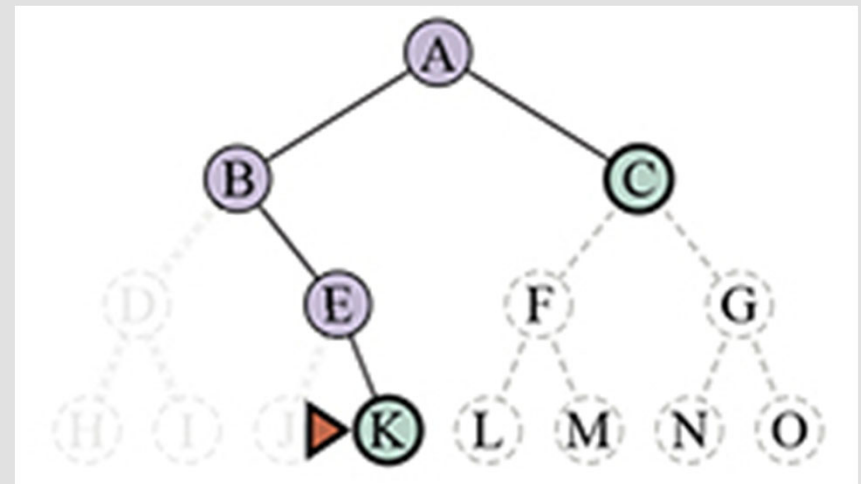
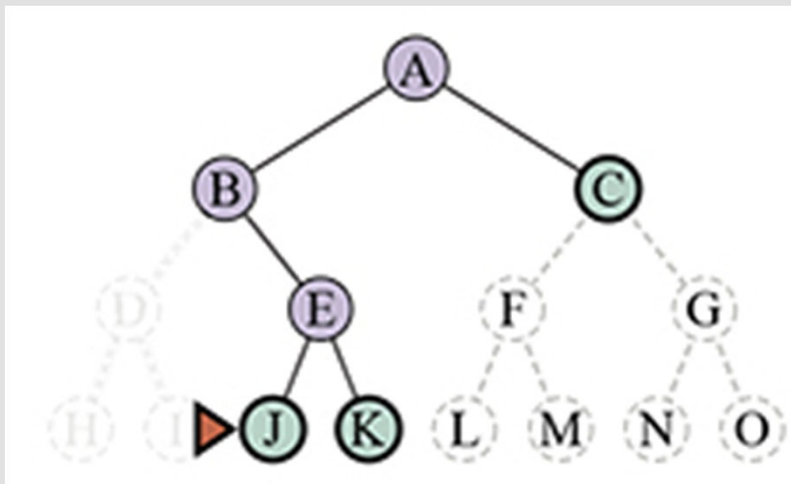
Depth-first Search (III)

- **Expand the deepest unexpanded node**
- **Implementation: managing the frontier**
 - QUEUEING-FN: LIFO – insert successors in front of queue



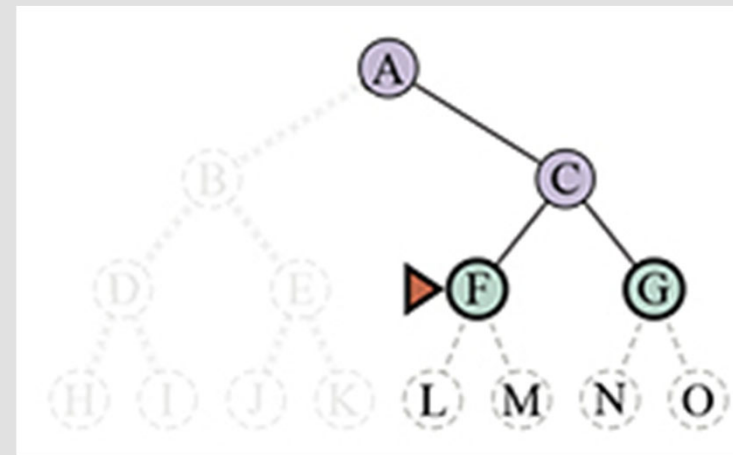
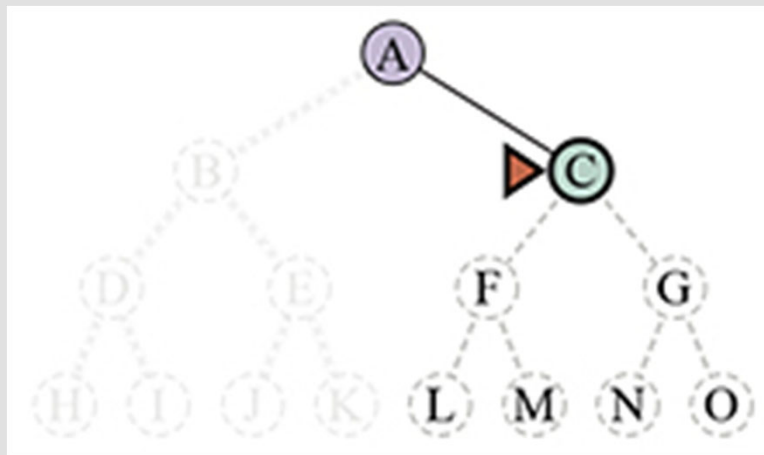
Depth-first Search (IV)

- **Expand the deepest unexpanded node**
- **Implementation: managing the frontier**
 - QUEUEING-FN: LIFO – insert successors in front of queue



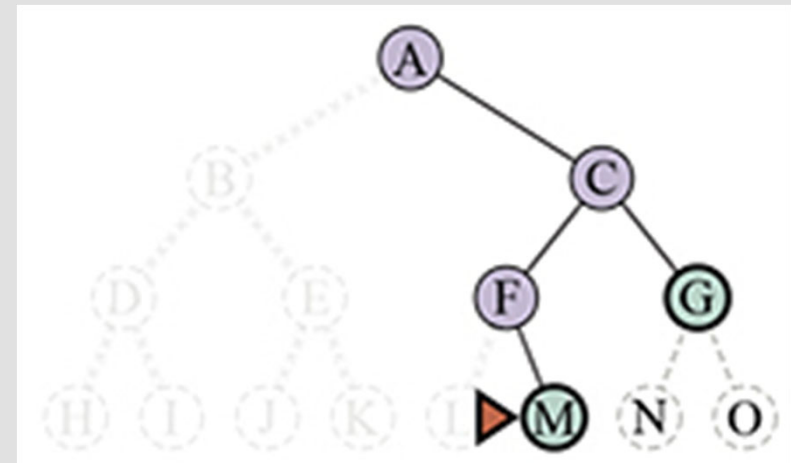
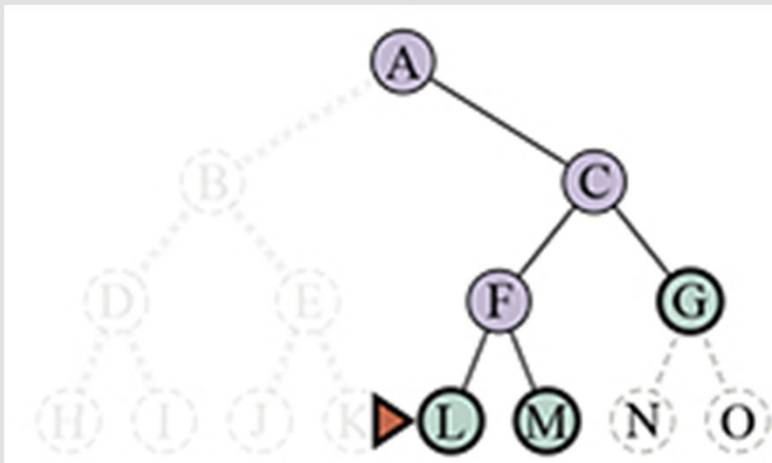
Depth-first Search (V)

- **Expand the deepest unexpanded node**
- **Implementation: managing the frontier**
 - QUEUEING-FN: LIFO – insert successors in front of queue



Depth-first Search (VI)

- **Expand the deepest unexpanded node**
- **Implementation: managing the frontier**
 - QUEUEING-FN: LIFO – insert successors in front of queue



Properties of Depth-first Search

- Complete?
 - Infinite-state spaces: No
 - Finite-state spaces: Yes, if we check for ancestors
- Time? $O(b^m)$, terrible if m is much larger than d
- Space? $O(bm)$, i.e., linear space
- Optimal? No

When all step costs are the same, will DFS find the optimal path?

Depth-limited Search

- **Depth-first search with depth limit L**
 - nodes at depth L have no successors
 - returns *cut-off* if no solution is found

```
function DEPTH-LIMITED-SEARCH(problem,  $\ell$ ) returns a node or failure or cutoff  
  frontier  $\leftarrow$  a LIFO queue (stack) with NODE(problem.INITIAL) as an element  
  result  $\leftarrow$  failure  
  while not IS-EMPTY(frontier) do  
    node  $\leftarrow$  POP(frontier)  
    if problem.IS-GOAL(node.STATE) then return node  
    if DEPTH(node) >  $\ell$  then  
      result  $\leftarrow$  cutoff  
    else if not IS-CYCLE(node) do  
      for each child in EXPAND(problem, node) do  
        add child to frontier  
  return result
```



Properties of Depth-limited Search

- Complete? Yes, if $d \leq L$
- Time? $b + b^2 + b^3 + \dots + b^L = b \frac{b^L - 1}{b - 1} \rightarrow O(b^L)$
- Space? $O(bL)$
- Optimal? No

When all step costs are the same, will DLS find the optimal path?



FIT3080 – Artificial Intelligence

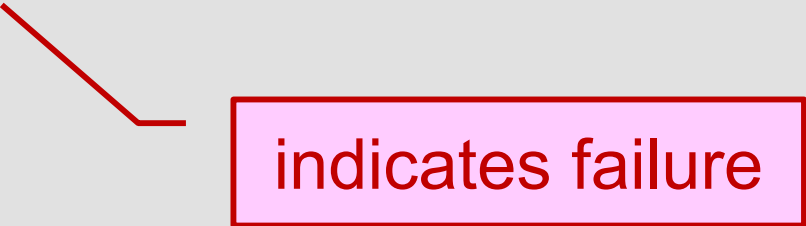
Iterative Deepening Depth-First Search

Iterative Deepening DF Search

- **Depth-limited search that gets repeated for every depth**

function ITERATIVE-DEEPENING-DF-SEARCH(*problem*) **returns** a solution or failure

- Initialize the frontier using the initial state of *problem*
- **For** $depth \leftarrow 0$ **to** ∞
 - $result \leftarrow$ DEPTH-LIMITED-SEARCH(*problem*, $depth$)
 - **if** $result \neq \text{cut-off}$ **then return** $result$
- **end**

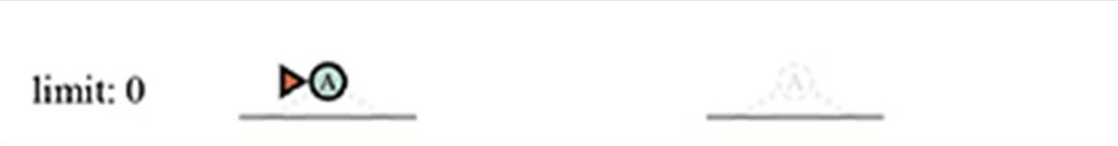


indicates failure



Iterative Deepening DF Search (I)

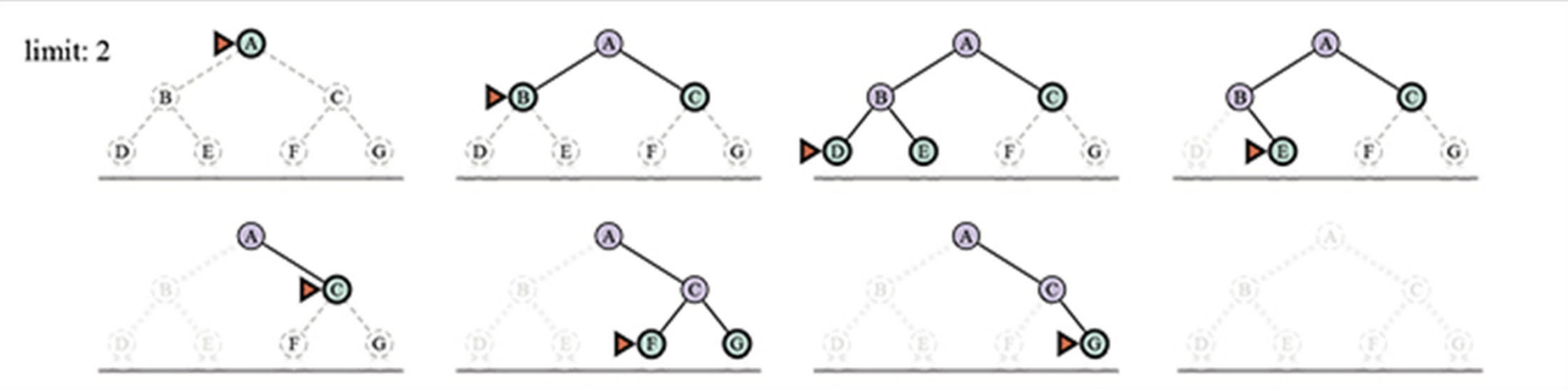
Depth 0



Depth 1



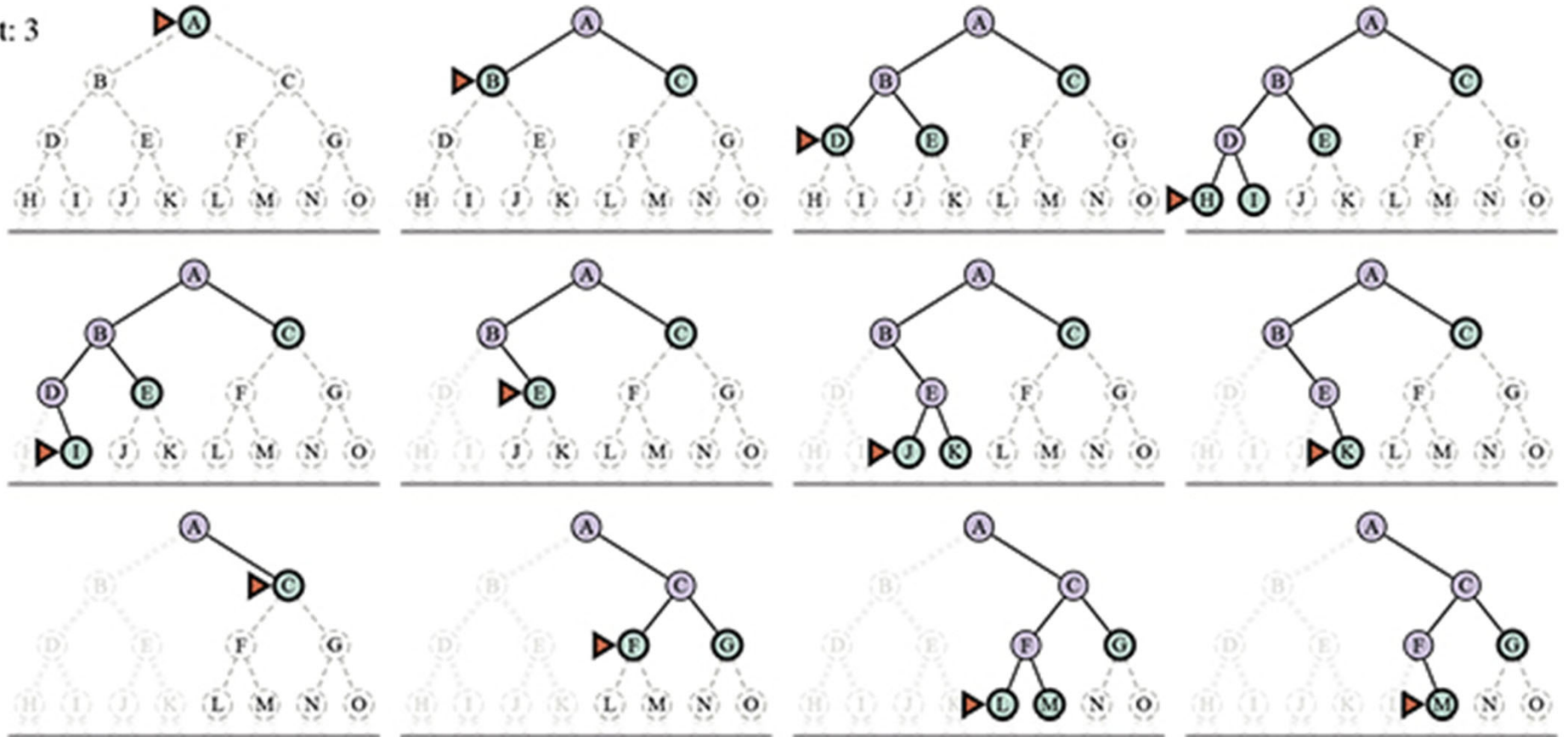
Depth 2



Iterative Deepening DF Search (II)

Depth 3

limit: 3



Iterative Deepening Search – Generated Nodes

- Number of nodes generated in a depth-limited search to depth d with branching factor b :

$$N_{DLS} = b + b^2 + b^3 + \dots + b^d = b \frac{b^d - 1}{b - 1} \rightarrow O(b^d)$$

- Number of nodes generated in an iterative deepening search to depth d with branching factor b :

$$N_{IDS} = db + (d - 1)b^2 + \dots + 3b^{d-2} + 2b^{d-1} + b^d \rightarrow O(b^d)$$

- Example: For $b = 10$, $d = 6$,

- $N_{DLS} = 1 + 10 + 100 + 1,000 + 10,000 + 100,000 = 111,111$

- $N_{IDS} = 6 + 50 + 400 + 3,000 + 20,000 + 100,000 = 123,456$

- Overhead = $\frac{123,456 - 111,111}{111,111} = 11\%$

Properties of Iterative Deepening Search

- Complete? Yes
- Time?
 $db + (d - 1)b^2 + \dots + 3b^{d-2} + 2b^{d-1} + b^d \rightarrow O(b^d)$
- Space? $O(bd)$
- Optimal? Yes, if step costs are identical

Uninformed TreeSearch Algorithms (I)

Criterion	Back track	BFS	UCS	DFS	DLS	IDS
Expansion	1 op at a time	FIFO ¹	Min. cost ¹	LIFO ¹	LIFO ¹	LIFO ¹
Deletes nodes	Yes	No	No	Yes	Yes	Yes
Has depth limit	Yes ²	No	No	No	Yes	No
Restarts	No	No	No	No	No	Yes

1. All children at once
2. A bound can be set



Uninformed TreeSearch Algorithms (II)

Criterion	BFS	UCS	DFS	DLS	IDS
Complete?	Yes ¹	Yes ²	Yes ³	Yes ⁴	Yes
Optimal?	Yes ⁵	Yes	No	No	Yes ⁵
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O(b^m)$	$O(b^L)$	$O(b^d)$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O(bm)$	$O(bL)$	$O(bd)$

b branching factor

m maximum depth of the search tree

d = depth of the shallowest solution or m when there is no solution

L depth limit

1. Complete if b is finite and the state space is finite
2. Complete if b is finite and all action costs $\geq \varepsilon > 0$
3. Complete if the state space is finite
4. Complete if $d \leq L$
5. Optimal if all step costs are identical

Reading

- **Russell, S. and Norvig, P., *Artificial Intelligence – A Modern Approach* (4th ed), Prentice Hall**
 - Sections 3.1-3.4.4, 3.4.6
 - Sections 5.1-5.4 – only backtrack algorithm



Next Lecture Topic

- **Lecture Topic 3b**
 - Problem Solving as Search – Informed Search

